

NLP-Based Fake Job Posting Detection Using TF-IDF and Machine Learning

A Mini NLP Project Report

1. Title

NLP-Based Fake Job Posting Detection Using TF-IDF and Machine Learning

2. Problem Statement

Online job portals receive a large number of job postings every day, and a small but significant fraction of these are fraudulent listings created to steal money, personal information, or financial details from job seekers under the guise of a genuine employment opportunity. Manually verifying every posting is not scalable for large platforms. The objective of this project is to automatically classify a given job posting as Real or Fraudulent by analyzing its textual content using Natural Language Processing (NLP) and machine learning techniques.

3. Objectives

- To study and apply NLP preprocessing techniques suitable for classifying unstructured job posting text.
- To convert raw text into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency).
- To build and compare machine learning classification models (Logistic Regression and Naive Bayes) for detecting fraudulent postings.
- To handle the significant class imbalance present in the dataset (fraudulent postings are a small minority).
- To evaluate model performance using appropriate metrics (Accuracy, Precision, Recall, F1-score, Confusion Matrix) rather than accuracy alone.
- To analyze the limitations of the approach and identify directions for future improvement.

4. Dataset

Dataset name:

Real / Fake Job Posting Prediction

Source:

Kaggle (uploaded by Shivam Bansal), originally compiled from the University of the Aegean's Employment Scam Aegean Dataset (EMSCAD).

Size:

17,880 job postings.

Important attributes/fields:

title, location, department, salary_range, company_profile, description, requirements, benefits, telecommuting, has_company_logo, has_questions, employment_type, required_experience, required_education, industry, function.

Target variable:

fraudulent (binary: 0 = Real, 1 = Fraudulent).

Number of classes:

2 (binary classification).

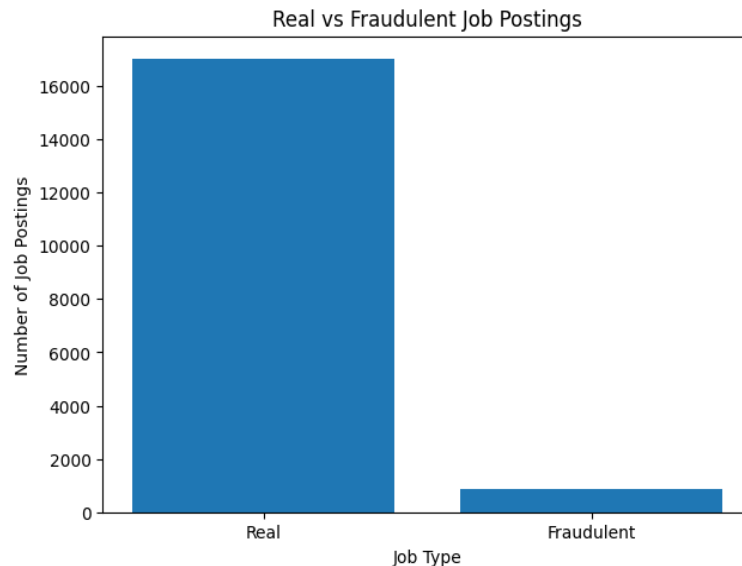


Figure 1: Class distribution of the dataset - 17,014 real postings (95.16%) vs 866 fraudulent postings (4.84%).

Data preparation:

- Missing values in text columns (company_profile, requirements, benefits, etc.) were filled with empty strings rather than dropping rows, to avoid losing genuine postings.
- Five text fields (title, company_profile, description, requirements, benefits) were combined into a single text field per posting, since fraud indicators can appear in any of these fields.
- Text was cleaned: lowercased, URLs and email addresses removed, digits and punctuation removed, tokenized, stopwords removed, and lemmatized.

Limitations of the dataset:

- Highly imbalanced - only 4.84% of postings are fraudulent, requiring special handling during model training and evaluation.
- Significant missing data in several fields (e.g., salary_range missing in a majority of rows).
- Contains English-language postings only, limiting generalization to other languages.
- Reflects scam patterns and language current at the time of data collection; newer scam tactics may not be represented.

5. Methodology

The overall approach followed in this project is:

Raw Job Posting Text → Preprocessing → TF-IDF Feature Extraction → Classification Model (Logistic Regression / Naive Bayes) → Prediction → Evaluation

Preprocessing:

Combines the relevant text fields, lowercases the text, removes URLs, email addresses, digits, and punctuation, then tokenizes, removes stopwords, and applies lemmatization to reduce words to their base form. This reduces noise and vocabulary size so the model can learn meaningful patterns rather than superficial variations.

TF-IDF Feature Extraction:

Converts each cleaned posting into a numerical vector. Term Frequency measures how often a word/phrase appears in a given posting; Inverse Document Frequency down-weights words that are common across most postings (carrying little discriminative signal) and up-weights words that are distinctive. Unigrams and bigrams (single words and two-word phrases) were used, with a maximum vocabulary size of 10,000 and a minimum document frequency of 2, to capture short but informative phrases (e.g., "wire transfer").

Classification Model:

Two models were trained and compared: Logistic Regression (with `class_weight="balanced"` to address class imbalance) and Multinomial Naive Bayes. Both are standard, well-understood algorithms for text classification with TF-IDF features.

Train/Test Split:

The dataset was split into 80% training (14,304 postings) and 20% testing (3,576 postings), using stratified sampling to preserve the same fraud ratio in both sets. The TF-IDF vectorizer was fit only on the training data and then used to transform the test data, to avoid data leakage.

Evaluation:

Model predictions on the held-out test set were compared against the true labels using Accuracy, Precision, Recall, F1-score, and a Confusion Matrix, with particular attention to the fraudulent class given the dataset's imbalance.

6. Implementation

Programming language:

Python 3.

Libraries used:

pandas and numpy (data handling), nltk (stopword removal, lemmatization), scikit-learn (TF-IDF vectorization, Logistic Regression, Multinomial Naive Bayes, evaluation metrics), matplotlib and seaborn (visualization), joblib (saving the trained model and vectorizer).

NLP techniques used:

Text cleaning (lowercasing, URL/email/digit/punctuation removal), tokenization, stopword removal, lemmatization, and TF-IDF vectorization with unigrams and bigrams.

Algorithm/model:

Logistic Regression (with balanced class weighting) and Multinomial Naive Bayes, compared against each other.

Important implementation details:

- `class_weight="balanced"` was used for Logistic Regression to penalize misclassification of the minority (fraudulent) class more heavily, without discarding any training data.
- The trained TF-IDF vectorizer and Logistic Regression model were saved using `joblib` for reuse without retraining.
- A `predict_job()` function was implemented to classify a brand-new, unseen job description and report the model's confidence (probability) for each class, demonstrating practical usability.

7. Results

The two models were evaluated on the same held-out test set of 3,576 postings. Results are summarized below:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9698	0.6360	0.8786	0.7379
Naive Bayes	0.9636	0.7471	0.3757	0.5000

Both models achieve high overall accuracy (>96%), which is expected given the class imbalance - a model predicting "Real" for every posting would already score above 95% accuracy. The more meaningful comparison is on the fraudulent class specifically: Logistic Regression achieves substantially higher Recall (87.9% vs 37.6%), meaning it correctly identifies far more of the actual fraudulent postings, at the cost of lower Precision (more false alarms on real postings). This trade-off is a direct result of `class_weight="balanced"`, which Naive Bayes does not support. Based on F1-score, Logistic Regression is the better-performing model overall (0.738 vs 0.500).

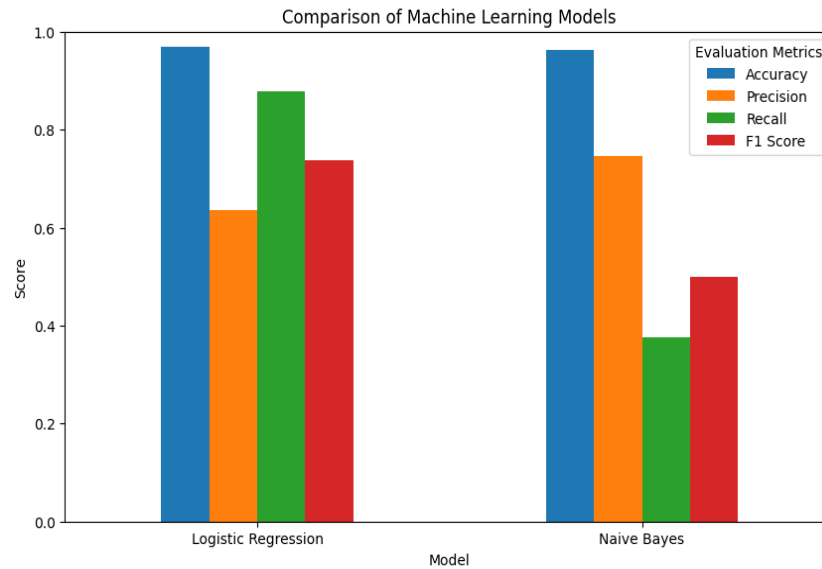


Figure 2: Comparison of Accuracy, Precision, Recall, and F1-score between Logistic Regression and Naive Bayes.

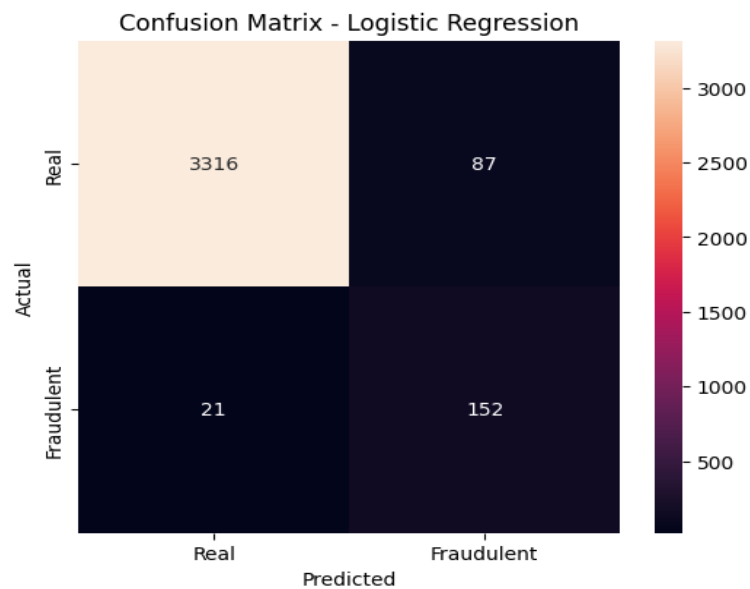


Figure 3: Confusion Matrix - Logistic Regression. The model correctly identified 152 of 173 fraudulent postings (21 missed) and 3,316 of 3,403 real postings (87 false alarms).

Sample output:

When tested on a new, unseen job description (a normal software developer posting), the Logistic Regression model correctly classified it as "Real Job Posting" with 83.4% confidence, demonstrating the model's practical applicability to new postings.

8. Limitations

- The dataset is highly imbalanced (only 4.84% fraudulent), which limits how precisely the minority class can be learned even with class weighting.
- TF-IDF is a bag-of-words style representation and does not capture word order, context, or sarcasm; a well-written fraudulent posting mimicking real corporate language may be missed.
- The model's vocabulary is limited to words seen during training (max 10,000 features); entirely new scam phrasing not present in the training data may not be detected.
- Performance may not generalize well to job postings in languages other than English, or to domains/industries underrepresented in the dataset.
- The dataset reflects scam patterns at the time of collection; fraud tactics evolve over time, so periodic retraining would be needed in a real deployment.

9. Future Scope

- Use a larger and more diverse dataset, including postings from multiple regions, languages, and time periods.
- Apply advanced deep learning models such as Bidirectional LSTM or Transformer-based models (e.g., BERT) to capture contextual meaning beyond TF-IDF's bag-of-words approach.
- Incorporate the dataset's structured (non-text) fields, such as `has_company_logo`, `telecommuting`, and `required_experience`, alongside text features.
- Add multilingual support to detect fraudulent postings in languages other than English.
- Deploy the model as a real-time web or browser-based tool that screens postings before publication.
- Use more advanced imbalance-handling techniques such as SMOTE oversampling, in addition to class weighting.
- Integrate the system into existing job portal pipelines as an automated pre-publication filter.

10. Conclusion

This project developed an NLP-based system to automatically detect fraudulent job postings using text classification. Job posting text was cleaned and preprocessed using standard NLP techniques (tokenization, stopword removal, lemmatization) and converted into numerical features using TF-IDF with unigrams and bigrams. Two machine learning models, Logistic Regression and Multinomial Naive Bayes, were trained and compared on a held-out test set of 3,576 postings from a dataset of 17,880 job listings. Logistic Regression, combined with balanced class weighting to address the dataset's significant class imbalance, achieved the better overall performance (F1-score of 0.738, Recall of 87.9% on the fraudulent class), correctly identifying the large majority of fraudulent postings while maintaining high overall accuracy (96.9%). The project demonstrated that a relatively simple, well-understood NLP pipeline - text preprocessing combined with TF-IDF and a classical machine learning classifier - can effectively address a practically relevant problem, while also highlighting the importance of evaluating imbalanced classification tasks using metrics beyond accuracy alone.